Experiment No: 10

Analyze Captured Packets for Potential Security Issues

Aim

To capture and analyze network packets using Wireshark and identify potential security issues such as unauthorized access attempts, clear-text credentials, suspicious communication, and other malicious network activities.

Theory

Packet analysis is the process of examining network packets exchanged between devices to understand network communication and identify abnormal or suspicious activities.

Wireshark is a network protocol analyzer that captures packets from a network interface and provides detailed information about protocols, IP addresses, ports, packet contents, and communication patterns.

During packet analysis, the following security issues can be identified:

1. **Clear-text credentials**
    1. Some protocols transmit usernames and passwords without encryption.
    2. Examples: HTTP, FTP, Telnet, and some legacy protocols.
    3. Such credentials can potentially be exposed to anyone able to capture the traffic.
2. **Unauthorized access attempts**
    1. Repeated connection attempts to different ports or services may indicate port scanning or unauthorized access attempts.
    2. Repeated failed authentication attempts can also be suspicious.
3. **Unencrypted communication**
    1. HTTP, FTP, and Telnet traffic can expose sensitive information.
    2. HTTPS and SSH provide encryption for their application traffic.
4. **Suspicious IP addresses**
    1. Communication with unexpected or unknown external systems should be investigated.
    2. An IP address alone, however, is not sufficient evidence of malicious activity.

5. **Port scanning**

    1. A host attempting connections to many ports or many systems in a short period may indicate scanning activity.

6. **Abnormal traffic patterns**

    1. Large numbers of packets, repeated connection failures, unusual protocols, or unexpected data transfers can indicate potentially suspicious behavior.

Common Wireshark Filters

Purpose	Wireshark Filter
HTTP traffic	http
FTP traffic	ftp
Telnet traffic	telnet
DNS traffic	dns
TCP traffic	tcp
UDP traffic	udp
Failed TCP connections	tcp.flags.reset == 1
HTTP POST requests	http.request.method == "POST"
Specific IP address	ip.addr == 192.168.1.10
Specific TCP port	tcp.port == 23
SYN packets	tcp.flags.syn == 1
SYN without ACK	tcp.flags.syn == 1 && tcp.flags.ack == 0

Note: Perform packet capture only on networks and systems you are authorized to monitor. For demonstrating clear-text credentials, use a controlled lab environment with test accounts.

Procedure

Part A: Capture Packets Using Wireshark

1. Install and open **Wireshark** on the Windows computer.
2. Start Wireshark with the required permissions.
3. Select the active network interface, such as:
    1. Wi-Fi
    2. Ethernet
4. Click **Start Capturing Packets**.
5. Generate normal network traffic by opening websites or communicating with a test machine.
6. Allow the capture to run for a few minutes.
7. Click **Stop** to stop packet capture.
8. Save the capture using:
   File → Save As → security_analysis.pcapng

Part B: Analyze Network Communication

1. Identify IP Addresses

Apply:

ip

Observe:

- Source IP
- Destination IP
- Protocol
- Packet length

Identify the systems communicating with each other.

2. Identify Clear-Text Protocols

Use:

http

Check whether HTTP traffic is present.

Select an HTTP packet and choose:

Right Click → Follow → TCP Stream

Observe the transmitted data.

Similarly, check:

ftp

and:

telnet

If authentication information is transmitted without encryption in a controlled lab capture, document that the protocol exposes sensitive information.

3. Search for Authentication Information

For an authorized lab capture, inspect application-layer packets using:

http.request.method == "POST"

Then examine the packet details and TCP stream.

Look for fields such as:

username
password
user
login

Do not use or disclose real credentials found in a capture.

4. Detect Possible Port Scanning

Use:

tcp.flags.syn == 1 && tcp.flags.ack == 0

Observe whether one source IP is attempting connections to many destination ports.

For example:

192.168.1.20 → 192.168.1.10:21
192.168.1.20 → 192.168.1.10:22
192.168.1.20 → 192.168.1.10:23
192.168.1.20 → 192.168.1.10:80
192.168.1.20 → 192.168.1.10:443

A large number of connection attempts to different ports in a short period **may indicate scanning**, but should be verified against legitimate network activity.

5. Identify Failed TCP Connections

Apply:

tcp.flags.reset == 1

A large number of TCP Reset packets may indicate:

- Closed ports
- Failed connections
- Application errors
- Possible scanning activity

Further investigation is required before classifying the traffic as malicious.

6. Analyze DNS Traffic

Apply:

dns

Check:

- Querying IP address
- Requested domain
- Response address
- Frequency of queries

Unexpected domains or unusually frequent DNS requests can be investigated further.

Optional Command-Line Analysis

Wireshark provides **TShark**, its command-line packet analyzer.

To display packets from a capture file:

tshark -r security_analysis.pcapng

To display HTTP packets:

tshark -r security_analysis.pcapng -Y "http"

To display DNS packets:

tshark -r security_analysis.pcapng -Y "dns"

To display TCP SYN packets:

tshark -r security_analysis.pcapng -Y "tcp.flags.syn == 1 && tcp.flags.ack == 0"

To display TCP reset packets:

tshark -r security_analysis.pcapng -Y "tcp.flags.reset == 1"

To display FTP traffic:

tshark -r security_analysis.pcapng -Y "ftp"

These commands are intended for analysis of a capture that you are authorized to examine.

Conclusion

The captured packets were successfully analyzed using **Wireshark/TShark**. Network protocols, IP addresses, ports, authentication-related traffic, TCP connection attempts, DNS queries, and other communication patterns were examined to identify potential security issues.